

CS102L Data Structures & Algorithms

Lab 8



DEPARTMENT OF COMPUTER SCIENCE
DHANANI SCHOOL OF SCIENCE AND ENGINEERING
HABIB UNIVERSITY
SPRING 2026

Contents

1	Introduction	2
1.1	Instructions	2
1.2	Marking scheme	2
1.3	Lab Objectives	2
1.4	Late submission policy	2
1.5	Use of AI	3
1.6	Viva	3
2	Prerequisites	4
2.1	Pytest	4
3	Exercises	5
3.1	Creating a Hash Table	5
3.1.1	Function Description	5
3.1.2	Examples	5
3.1.3	Testing	5
3.2	Creating a Hash Function	5
3.2.1	Function Description	5
3.2.2	Examples	6
3.2.3	Testing	6
3.3	Search, Get and Put	6
3.3.1	Function Description	7
3.3.2	Search	7
3.3.3	Get	7
3.3.4	Put	8
3.3.5	Examples	8
3.3.6	Testing	8
3.4	Delete	8
3.4.1	Function Description	9
3.4.2	Examples	9
3.4.3	Testing	9
3.5	Practically Checking the Hash Table	9
3.5.1	Function Description	10
3.5.2	Examples	10
3.5.3	Testing	10

Introduction

1.1 Instructions

- This lab will contribute 1% towards your final grade.
- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *lab8-aa12345.zip* where *aa12345* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 100.

- 50 Marks are for the completion of the lab.
- 50 Marks are for progress and attendance during the lab.

1.3 Lab Objectives

- To understand and implement hashing in Python
- To use hashing in different exercises

1.4 Late submission policy

There is no late submission policy for the lab.

1.5 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.6 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

In case the above mentioned pytest commands do not work on your system, even though pytest is installed, try using the following commands:

```
py -m pytest
```

```
py -m pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
===== test session starts =====
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py .....

===== 10 passed in 0.06s =====
```

However, if all the tests fail, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
===== test session starts =====
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFF

===== FAILURES =====
test_question1[1st10-1st20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] ____
```

Exercises

3.1 Creating a Hash Table

This function should be implemented in the file **q1.py**.

3.1.1 Function Description

Write a function called `create_hashtable` that creates a hash table and returns it. You will represent your hash table as a tuple with two lists in it. The first list represents the keys and the second list represents their associated values. Keys and values should be stored at the same index in both of the lists i.e., a key at index 1 in the first list will have its associated value at index 1 in the second list. This function takes a variable called `size` which would be the size of the hash table i.e., the size of both of the lists inside the tuple.

3.1.2 Examples

```
>>> size = 4
>>> print(create_hashtable(size))
([None, None, None, None], [None, None, None, None])
```

3.1.3 Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q1.py
```

3.2 Creating a Hash Function

This function should be implemented in the file **q2.py**.

3.2.1 Function Description

A hash function is a function that gives the hash of a given key. This hash is used as an index in hash tables. You will create the following two functions:

- `hash_function(key,size)`
 - Here, the key represents the key out of a key-value pair and size is the size of the hash table.
 - If the key is of type integer, use the Divide Modulo function, i.e. $\text{key} \% \text{size}$ to generate and return the index where the key and its value should be placed.
 - If the key is of type string, then the index will be calculated by the sum of the Unicode/ASCII values of each letter in the string. Don't forget to get the modulus value so that it remains within the range of the hash table size.
- `collision_resolver(key, size, iteration)`
 - Use Linear Probing to resolve collisions
 - Here, the key represents the key out of a key-value pair, size is the size of the hash table, and iteration is the iteration number to get the next interval in case of collisions
 - Get the new hash value by getting the original hash value using `hash_function` and then adding *iteration* to the original hash value and taking a mod of it with size.

3.2.2 Examples

```
>>> key = "Hello"
>>> size = 11
>>> iteration = 2
>>> hash_value = hash_function(key, size)
>>> hash_value
5
>>> print(collision_resolver(key, size, iteration))
7
```

3.2.3 Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q2.py
```

3.3 Search, Get and Put

This function should be implemented in the file **q3.py**.

3.3.1 Function Description

A hash table would need a getter and setter function which would place the key-value pair in their appropriate location and get the value of a key when called. Create the following three functions to implement the primary getter and setter operations of hashing:

3.3.2 Search

Pseudocode 1 Search Function

Require: *hashtable*: hash table, *key*: key to find, *size*: size of the hash table

Ensure: Return the index if *key* is found and not deleted; otherwise return **nil**

```
1: Compute initial slot index:  $index = hash\_func(key) \bmod size$ 
2: Save this initial index as start to detect full cycle
3: while slot at index is not empty do
4:   if key at index matches key then
5:     return index
6:   else
7:     move to the next slot:  $index = (index + 1) \bmod size$ 
8:     if  $index = start$  then
9:       break {Full table searched}
10:    end if
11:  end if
12: end while
13: return nil
```

3.3.3 Get

Pseudocode 2 Get Function

Require: *hashtable*: hash table, *key*: key to find, *size*: size of the hash table

Ensure: Return the entry if *key* is found and not deleted; otherwise return **nil**

```
1: if key exists in hashtable then
2:   return entry at index
3: else
4:   return nil
5: end if
```

3.3.4 Put

Pseudocode 3 Put Function

Require: *hashtable*: hash table, *key*: key to add, *value*: value to add, *size*: size of the hash table

Ensure: Insert key-value pair if key does not exist; otherwise, update the value of the existing key

```
1: if key exists in hashtable then
2:   Update the value of the key
3: else
4:   Find the empty slot. A slot is empty if hashtable[slot]  $\in \{\text{None}, \#\}$ 
5:   if hashtable is Full then
6:     Key cannot be added in hashtable
7:   end if
8:   if empty slot is available then
9:     Insert key and value into the slot
10:  end if
11: end if
```

3.3.5 Examples

```
>>> H = create_hashtable(5)
>>> put(H, "Hello", "World", 5)
>>> print(search(H, "Hello", 5))
0
>>> print(get(H, "Hello", 5))
World
>>> print(search(H, "World", 5))
None
>>> print(get(H, "World", 5))
None
```

3.3.6 Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q3.py
```

3.4 Delete

This function should be implemented in the file **q4.py**.

3.4.1 Function Description

Deleting a key value pair from a hash table is a critical part in the representation of a hash table. Create the function **delete** which remove a key value pair from a hash table if it is present in hashtable, and in place of it places a tombstone (the hashtag symbol: #) which signifies that something has been deleted and this index was not originally empty.

Pseudocode 4 Delete Function

Require: *hashtable*: hash table, *key*: key to remove, *size*: size of the hash table

Ensure: Remove key-value pair if key exists.

- 1: **if** *key* exists in *hashtable* **then**
 - 2: Replace the *key* and its corresponding value with a tombstone marker (#).
 - 3: **end if**
-

3.4.2 Examples

```
>>> H = create_hashtable(5)
>>> put(H, "Hello", "World", 5)

>>> delete(H, "World", 5)
>>> print(H)
(['Hello', None, None, None, None], ['World', None, None,
None, None])

>>> delete(H, "Hello", 5)
>>> print(H)
(['#', None, None, None, None], ['#', None, None, None, None])

>>> delete(H, "Hello", 5)
>>> print(H)
(['#', None, None, None, None], ['#', None, None, None, None])
```

3.4.3 Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q4.py
```

3.5 Practically Checking the Hash Table

This function should be implemented in the file **q5.py**.

3.5.1 Function Description

Create a function called `main` that simulates how a hash table works. It will firstly take as a parameter the size of a hash table. It would also take the following 3 lists as parameters:

- `to_put` - This is a list of tuples where each tuple contains a key value pair. These are to be added to a hash table.
- `to_delete` - This is a list of keys whose associated key value pair has to be deleted from the hash table.
- `to_get` - This is a list of keys whose values are to be retrieved and stored in a list.

You will first create a hash table based on the given size and then do the necessary operations based on the available data. This function should return a list of values which are associated with the keys in the `to_get` list. It should also return the hash table itself as well. These should be in a tuple that would look like: (values to get, hash table).

3.5.2 Examples

```
>>> size = 5
>>> to_put = [(1,2),("key","value")]
>>> to_delete = [1]
>>> to_get = ["key"]
>>> print(main(to_put, to_delete, to_get, size))
(['value'], ([None, '#', None, 'key', None], [None, '#',
None, 'value', None]))
```

3.5.3 Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q5.py
```